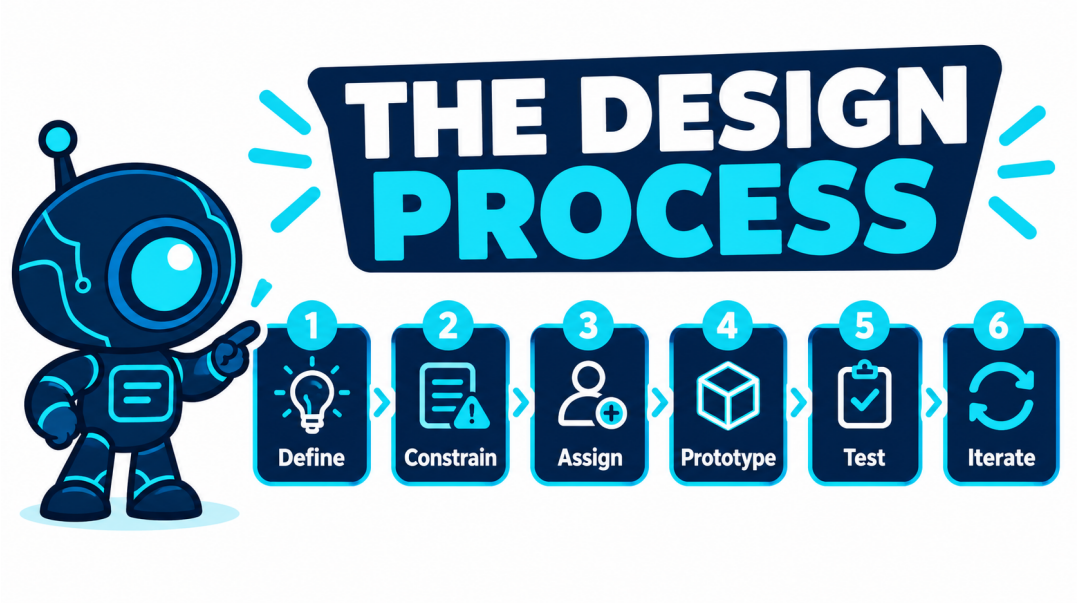
AI MECH LAB

Mechatronics Course — Beginner Track



Module 1, Topic 1.3

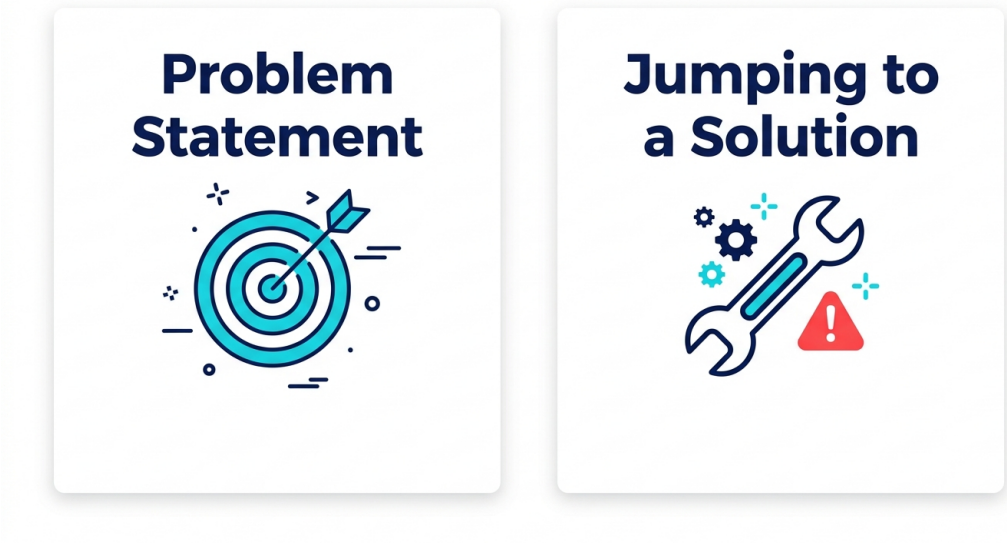
The Mechatronics Design Process

Lesson

You've now seen the four pillars and how they show up in real systems. This lesson zooms out one more level: how does a mechatronic system actually get built in the first place? Every elevator, CNC machine, or drone you looked at last lesson started as a problem someone needed to solve, and ended as a working machine. The path between those two points follows a repeatable process — and knowing that process is what separates building something on purpose from building something by accident.

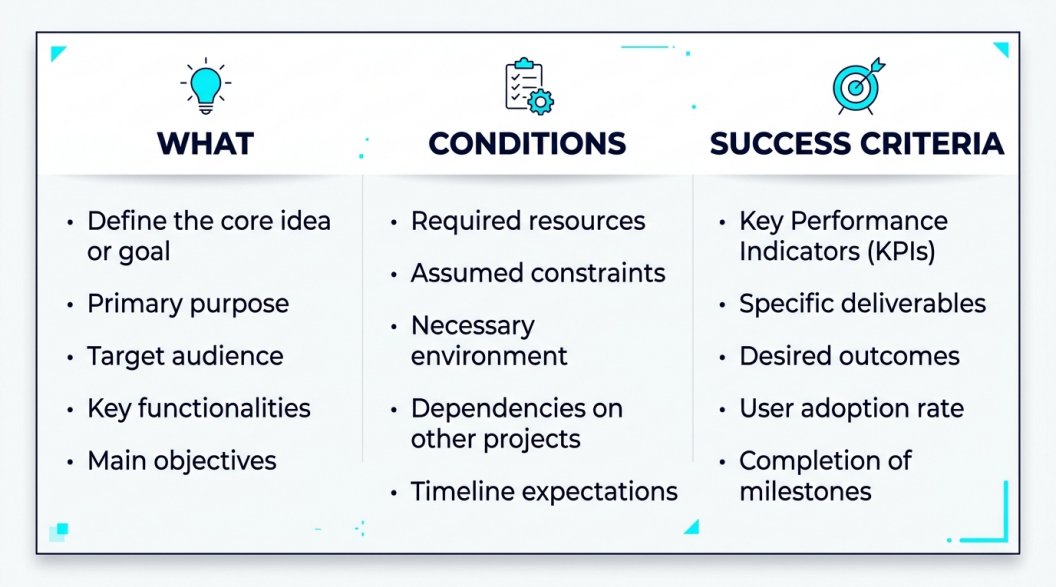
Step 1 — Define the problem, not the solution

Every mechatronic project starts with a clear problem statement, not a component list. "Build a system that stops a conveyor belt automatically if an object gets too close to the edge" is a problem statement. "Use an ultrasonic sensor and a relay" is already a solution — and jumping straight to a solution before the problem is fully defined is one of the most common ways projects go wrong.



A clear target versus reaching for tools before the target is set.

A good problem statement answers three things: what needs to happen, under what conditions, and what counts as success.



The three parts every problem statement needs to answer.

For the conveyor example: the belt must stop (what), when an object crosses a defined distance threshold near the edge (conditions), within a response time fast enough to prevent the object falling (success).

Step 2 — Set constraints before you design anything

Before sketching a single part, define what you're working within: budget, available components, size limits, power source, environment (indoor/outdoor, dusty, wet), and how much time you have. Constraints aren't obstacles to work around later — they actively shape which design choices even make sense.



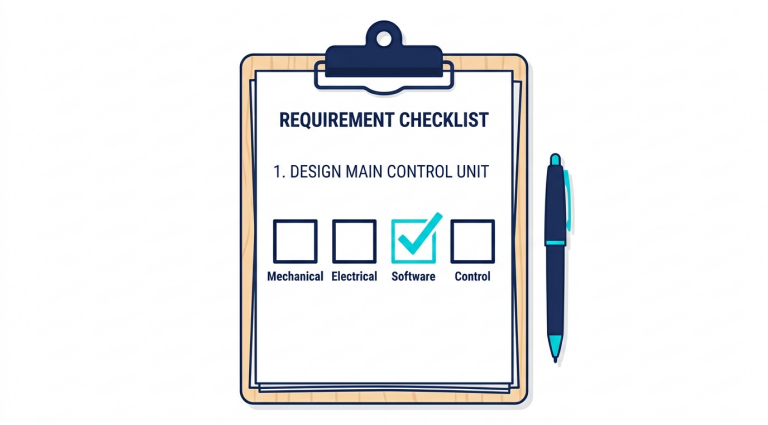
The constraints worth defining before any design work begins.

A system that needs to run on battery power for a week rules out a lot of otherwise-reasonable component choices before you've drawn a single diagram. Skipping this step is how people end up designing something impressive that's completely unbuildable with what they actually have access to.

Step 3 — Assign the four pillars

This is where the four pillars from Topic 1.1 stop being a concept and start being a planning tool. For each requirement in your problem statement, decide which pillar handles it: what needs a physical

mechanism (mechanical), what needs power or sensing (electrical), what needs decision logic (software), and what needs a feedback loop to keep things accurate (control).

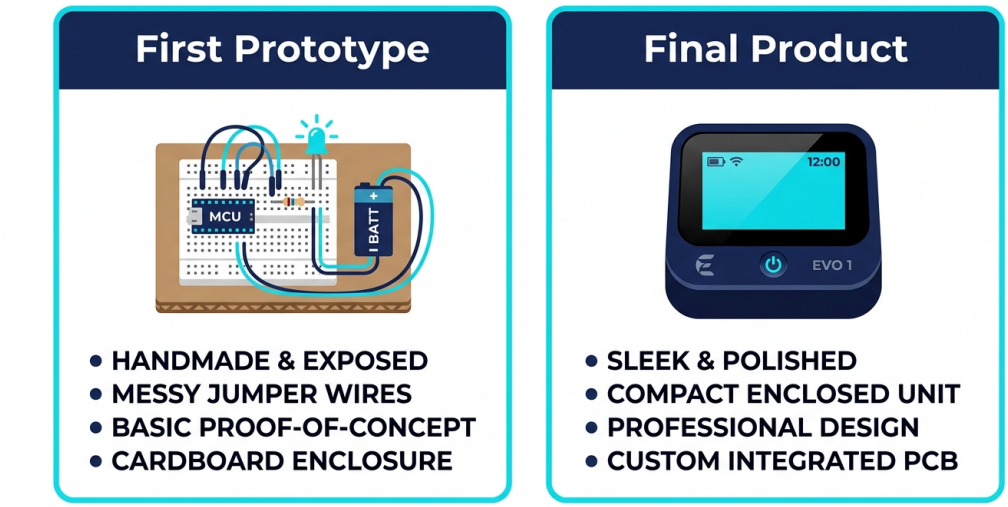


Sorting one requirement into the pillar responsible for it.

Doing this explicitly, on paper, before building anything, catches gaps early — like realizing you've planned the sensor and the motor but never decided what logic connects the two.

Step 4 — Build a rough prototype, not a finished product

The first version you build should be the cheapest, fastest way to test whether your core idea actually works — not a polished final product. Cardboard instead of machined housing. A breadboard instead of a custom circuit board.

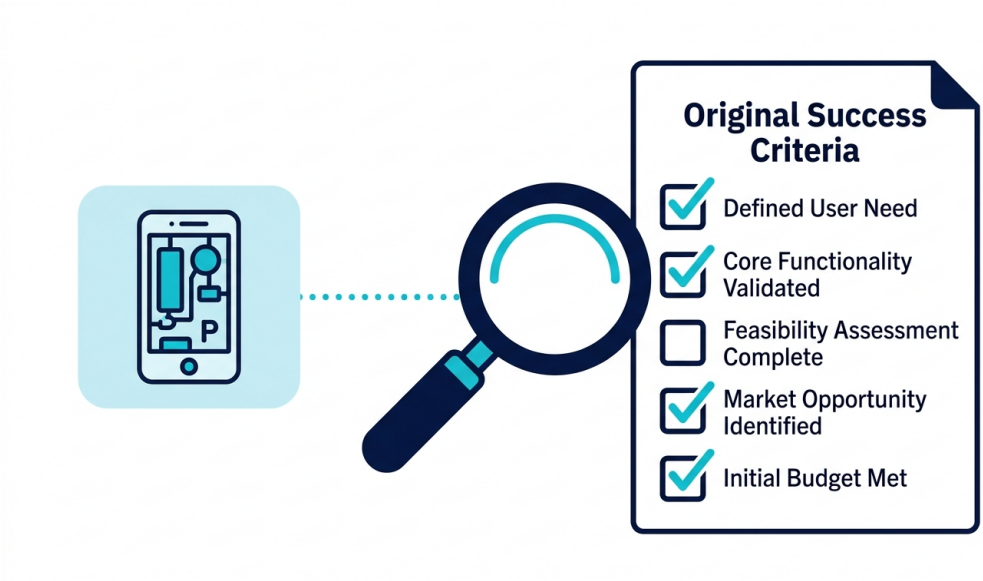


A rough first pass and a finished product serve two different purposes.

The goal of a first prototype is answering one question: does the fundamental approach work at all? Everything else — appearance, durability, cost efficiency — comes after that question is answered, not before. Building a polished version first, before knowing the core idea works, means any flaw you find costs far more time and material to fix than it would have at the rough-prototype stage.

Step 5 — Test against your original success criteria

Go back to the problem statement from Step 1 and test the prototype against exactly what you wrote down. Not "does it seem to work" — does it meet the specific conditions and success criteria you defined at the start?

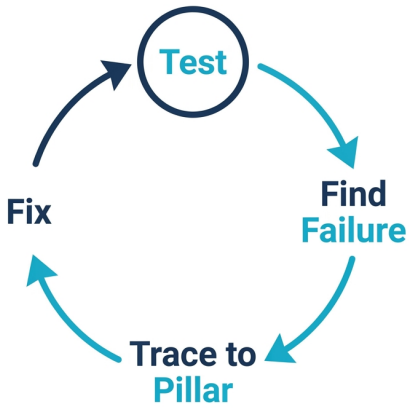


Checking the built prototype against the list written before any building began.

This is where a clear Step 1 pays off: vague problem statements make vague testing possible, and vague testing is how systems ship with real gaps nobody noticed.

Step 6 — Iterate based on what actually failed

No first prototype meets every criterion. The design process is a loop, not a straight line: test, find what failed, trace that failure back to a pillar, fix it, test again.

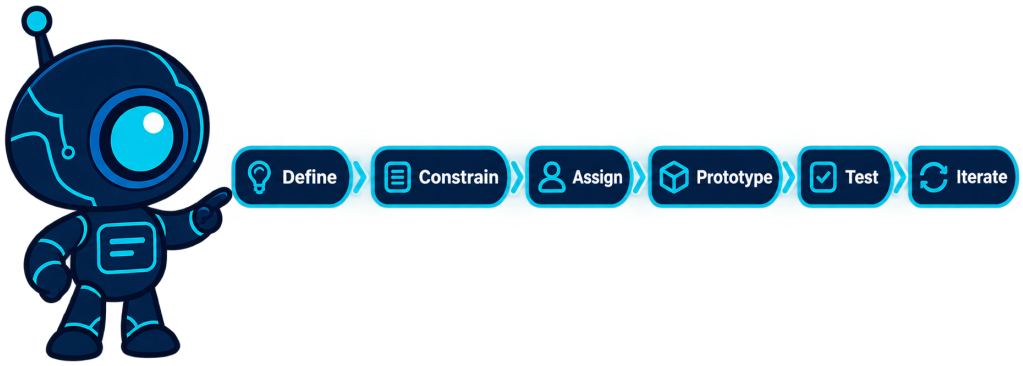


The loop every prototype runs through on the way to working reliably.

A late stop time might be a control problem (the feedback loop reacts too slowly) or a mechanical problem (the belt itself can't stop fast enough no matter what the control system does) — and knowing which pillar actually caused the failure is what keeps you from "fixing" the wrong part of the system.

Where this leaves you

By the end of this lesson, you should be able to walk through the same six-step process for any mechatronic problem: define the problem clearly, set your constraints, assign the four pillars, build a rough prototype, test against your original criteria, and iterate based on what specifically failed. This process is what you'll be applying, in miniature, throughout the rest of this course — and at full scale, on real projects, after it. With Module 1 complete, you move next into Basic Electronics, where you start working with the actual electrical pillar hands-on: voltage, current, and resistance.



The six-step design process, start to finish.